REAL EVIDENCE RESEARCH REPORT: SOLVENT ETCD DATASET INTEGRATION

Executive Recommendation

To achieve an authoritative, deterministic, and judge-comprehensible demonstration of the Solvent Transactional Belief Ledger without expanding the frozen core architecture, Solvent must integrate a minimal pinned dataset composed exclusively of verified, real-world etcd security advisories and official release records[1].

Rather than attempting to ingest large vulnerability databases or depend on live network APIs, the optimal solution is a static, offline dataset containing two primary real-world security and reliability events, three release metadata records, and five supporting normalized evidence records[1]. This dataset fits entirely within a single pinned snapshot fixture of under 15 kilobytes, guaranteeing immediate local execution and complete offline reproducibility[1].

The primary real-world records selected for this integration are:

1. GitHub Security Advisory GHSA-q8m4-xhhv-38mg (CVE-2026-33413), which documents gRPC API authorization bypasses affecting etcd versions prior to 3.5.28[2].
2. The official etcd v3.5 Data Inconsistency Postmortem (v3.5-data-inconsistency.md) and associated issue tracking (#13654), which documents critical data corruption in etcd releases v3.5.0 through v3.5.2[4].
3. Official GitHub release records for etcd versions v3.5.0, v3.5.27, and v3.5.28[2].

Factual authenticity must be prioritized over artificial scenario completeness[1]. Real-world vulnerability lifecycles naturally exhibit sequential retraction and invalidation—where new advisories or vendor postmortems override previous assumptions of security—but rarely present simultaneous logical contradictions within the exact same ingestion tick[1].

Consequently, the selected dataset naturally supports Normal Promotion (Scenario A), Deterministic Replay (Scenario B), and Transactional Falsification/Retraction via RetractCascade (Scenario D) using 100% authentic real-world evidence[1]. To demonstrate Contradiction Detection (Scenario C) without fabricating false historical events, a clearly demarcated synthetic record representing a conflicting vendor claim is incorporated alongside the real records[1].

The end-to-end processing pipeline operates deterministically across Solvent's four database tables (evidence, belief, belief_edge, and action_intent)[1]. Ingested snapshot records are first written to the evidence table with cryptographic content hashes (SHA-256) and full provenance[1]. Normalization routines parse these raw records to derive semantic claims in the belief table and edge relationships in belief_edge[1]. Outstanding evidence debt items are accumulated and subsequently retired as supporting records arrive[1]. Once all required debt conditions are satisfied, a CockroachDB transaction promotes the belief state from DERIVED to PROMOTED, which legally permits the execution of associated action intents[1]. Should

invalidating evidence arrive later, Solvent executes a transactional RetractCascade, revoking the promoted belief and cancelling the dependent action intent while preserving a fully auditable immutable history[1].

Canonical Demo Narrative

The demonstration narrative is designed to complete in under two minutes, leading an evaluator seamlessly from raw evidence ingestion to transactional belief promotion and automated safety retraction[1]. The timeline below details the sequential progression of the demonstration across six distinct operational stages[1].

Stage	Duration	Primary Action	System Mechanics	Visual / Output Outcome
1. Evidence Ingestion	00:00 – 00:30	Offline snapshot ingestion[1]	Ingests GHSA-q8m4-xhhv-38mg and release metadata[1]. Writes raw payloads and SHA-256 hashes to evidence table[1].	Evaluator observes normalized evidence records created with full source provenance[1].
2. Belief & Debt Creation	00:30 – 00:50	Semantic claim derivation[1]	Derives belief "etcd v3.5.27 contains CVE-2026-33413"[2]. Accumulates Fix Availability Debt[1].	Belief status set to DERIVED. Action intent act_block_v3527 initialized as PENDING[1].
3. Promotion & Intent Gate	00:50 – 01:10	Debt satisfaction & promotion[1]	Ingests v3.5.28 release metadata[1]. Retires debt; executes CockroachDB	Belief transitions to PROMOTED. Action intent act_block_v3527 transitions to

			transaction promoting belief[1].	LEGAL[1].
4. Contradiction Ingestion	01:10 – 01:30	Conflicting claim ingestion[1]	Ingests vendor override assertion[1]. Derives CONTRADICTS edge in belief_edge[1].	System blocks unsafe action intent mutation while conflict remains active[1].
5. Transactional Retraction	01:30 – 01:50	Invalidation evidence processing[1]	Ingests v3.5-data-inconsistency.md[4]. Triggers RetractCascade in database transaction[1].	Promoted belief set to RETRACTED. Dependent live intent immediately updated to CANCELLED[1].
6. Audit Log Verification	01:50 – 02:00	Audit trail inspection[1]	Queries CockroachDB audit ledger for historical state transitions and edge graphs[1].	Demonstrates complete immutable record showing why action was safely blocked[1].

The evaluation begins as the user executes the CLI to ingest the pinned local JSON snapshot containing release metadata for etcd v3.5.27 and security advisory GHSA-q8m4-xhhv-38mg[1]. Solvent normalizes these records, populating the evidence table with source URLs, publication timestamps, and cryptographic payload hashes[1].

From this evidence, Solvent derives a belief that etcd v3.5.27 suffers from a high-severity gRPC authorization bypass[2]. Simultaneously, the system attaches an evidence debt item specifying that a remediation action cannot be authorized until an official fixed release version is verified in the evidence ledger[1].

When the CLI ingests release metadata for etcd v3.5.28, Solvent links this evidence to the fix requirement, retiring the outstanding debt[1]. Solvent then executes an atomic CockroachDB transaction that updates the belief status to PROMOTED and unlocks the action intent (Block-Deployment etcd v3.5.27), transitioning its gating state from PENDING to LEGAL[1].

To demonstrate real-world safety guards, the scenario introduces postmortem evidence (v3.5-data-inconsistency.md) documenting that early v3.5 release builds suffer from uncommitted state corruption[4]. This invalidating evidence triggers Solvent's transactional RetractCascade mechanism[1]. Within a single database transaction, the underlying stability belief is marked RETRACTED, cascading down the belief graph to flip the active deployment intent from LEGAL to CANCELLED[1]. The evaluator completes the demonstration by running a falsification check, confirming that while stale or unsafe actions were prevented from executing, the ledger maintains an immutable audit trail of every evidence transition, belief state, and graph edge[1].

Source Inventory

Establishing high technical credibility requires selecting evidence exclusively from primary sources that offer machine-readable structures, clear open-source licensing, and absolute temporal stability[1]. The table below evaluates candidate data sources across these operational metrics[1].

Source Name	Source Type	URL / Endpoint	License	Update Frequency	Selected Role in Dataset
GitHub Security Advisory (GHSA)	Primary	github.com/advisories/GHSA-q8m4-xhhv-38mg	CC-BY-4.0[8]	Real-time	Primary Advisory Evidence for CVE-2026-33413[2]
etcd Official Postmortems	Primary	github.com/etcd-io/etcd/blob/main/Documentation/postmortems/v3.5-data-inconsistency.md	Apache-2.0	Historical Static	Primary Invalidation Evidence for Release Stability[4]
etcd GitHub Releases	Primary	github.com/etcd-io/etcd/releases/tag/v3.5.28	Apache-2.0	Per-Release	Primary Version Metadata & Fix

					Verification[2]
OSV.dev Database	Secondary (Derived)	osv.dev/GHSA-q8m4-xhhv-38mg	CC-BY-4.0[8]	Real-time Sync	Rejected as redundant to direct GHSA JSON[1]
National Vulnerability Database (NVD)	Secondary	nvd.nist.gov/vuln/detail/CVE-2026-33413	Public Domain	Periodic Sync	Rejected due to slow sync and API rate limits[1]

Primary sources were chosen over secondary aggregators to guarantee authoritative authenticity[1]. Direct GitHub Security Advisories provide pristine JSON payloads containing precise semver ranges and vulnerability identifiers[2]. Official etcd project documentation—specifically maintainer postmortems checked directly into the etcd-io/etcd repository—serves as the ultimate authority regarding operational defects and software retractions[4]. Secondary aggregators such as NVD and OSV.dev, while valuable for broader enterprise scanning, introduce synchronization delays and redundant payload wrapping that add overhead without enhancing the core demonstration narrative[1].

Recommended Minimum Dataset

The minimum defensible dataset consists of exactly two security and reliability records, three release records, and five corresponding normalized evidence items[1]. This compact footprint ensures that an evaluator can inspect and comprehend every individual record during a short demonstration[1].

The selected records fulfill specific structural roles within the belief ledger lifecycle:

1. **Security Advisory Record GHSA-q8m4-xhhv-38mg (CVE-2026-33413)**: Serves as the primary security evidence record[2]. It documents gRPC API authorization bypasses affecting etcd versions prior to 3.5.28, establishing exact semver bounds (>=3.5.0, <3.5.28) and detailing specific protocol impacts across MemberList, Alarm, Lease, and Compaction endpoints[2].
2. **Postmortem Record v3.5-data-inconsistency.md (Issue #13654)**: Serves as the primary invalidation evidence record[4]. Authored by etcd maintainers, it establishes that early v3.5 releases (v3.5.0 through v3.5.2) suffer from non-atomic consistent index writes during high-load crashes, leading to silent state divergence across cluster members[4].
3. **Official Release Metadata (v3.5.0, v3.5.27, v3.5.28)**: Establishes the concrete target version topology[2]. Version v3.5.0 represents the initial release later invalidated by postmortem data[3]; version v3.5.27 represents the vulnerable production deployment

target[2]; and version v3.5.28 represents the verified patch release required to satisfy evidence debt[2].

Candidate records that did not contribute directly to the linear narrative were omitted[1]. For example, GHSA-2xhq-gv6c-p224 (CVE-2020-15114) was excluded because gateway proxy loops involve complex network routing configurations that distract from core database state logic[10]. Similarly, GHSA-x35m-3gp4-4fh5 (CVE-2026-44283) was rejected to eliminate structural redundancy, as GHSA-q8m4-xhhv-38mg already establishes gRPC authorization bypass mechanics with identical patch version thresholds[2].

Exact Records to Capture

The table below specifies the exact fields extracted from authoritative sources and pinned within the offline local fixture[1].

Record Identifier	Source Canonical URL	Publication Date	Key Extracted Fields	Fixture Demonstration Role
GHSA-q8m4-xhhv-38mg	github.com/advisories/GHSA-q8m4-xhhv-38mg	2026-03-20[2]	summary, severity, vulnerable_version_range, fixed_in	Primary Security Evidence[2]
POSTMORTEM-v3.5-CORRUPT	github.com/etcd-io/etcd/blob/main/Documentation/postmortems/v3.5-data-inconsistency.md	2022-04-20[4]	status, impact, affected_versions, fixed_version	Invalidation Evidence[4]
REL-ETCD-v3.5.0	github.com/etcd-io/etcd/releases/tag/v3.5.0	2021-06-16[4]	tag_name, published_at, target_commitish	Baseline Release Evidence[3]
REL-ETCD-v3.5.27	github.com/etcd-io/etcd/releases/tag/v3.5.27	2026-02-10[2]	tag_name, published_at, target_commiti	Affected Target Release Evidence[2]

			sh	
REL-ETCD-v3.5.28	github.com/etcd-io/etcd/releases/tag/v3.5.28	2026-03-20[2]	tag_name, published_at, target_commitish	Remediation Fix Release Evidence[2]

Evidence Provenance

Preserving strict cryptographic and historical provenance for every ingested record is essential to guarantee ledger auditability[1]. To optimize fixture size without sacrificing verifiability, provenance metadata is structured across three functional tiers[1].

The first tier encompasses attributes strictly required by the Solvent core engine[1]. These include the canonical source_url, the upstream source_id (e.g., GHSA-q8m4-xhhv-38mg), the categorical source_type (e.g., GITHUB_ADVISORY, ETCD_POSTMORTEM), and a content_hash calculated via SHA-256 directly over the pre-normalized raw JSON payload[1]. These fields ensure unique record identification, drive duplicate detection during replay, and support cryptographically verified audit trails[1].

The second tier includes metadata attributes that enrich the hackathon demonstration without impacting database constraints[1]. These comprise the ISO-8601 publication_date from the original publisher, the snapshot retrieval_date, and formal publisher_attribution identifiers (e.g., etcd-io, GitHub Security Advisory Database)[1]. These fields provide vital context when displaying belief timelines to evaluators[1].

The third tier consists of unnecessary operational metadata that is intentionally discarded during snapshot capture[1]. This includes raw HTML rendering tags, layout CSS, navigation links, third-party blog mirrors, and transient HTTP response headers[1]. Stripping this noise maintains fixture compactness and prevents non-deterministic hash shifts caused by cosmetic upstream website updates[1].

Raw Record → Solvent Evidence Mapping

Normalizing raw upstream records into Solvent's database schema requires mapping structured source JSON fields directly into the evidence table[1]. The table below illustrates how raw fields from GitHub Security Advisory GHSA-q8m4-xhhv-38mg map into Solvent's core schema columns[1].

Raw Upstream Source Field	Normalized Target Column	Injected Value / Data Transformation	Architectural Purpose

raw.ghsa_id	evidence.source_id	"GHSA-q8m4-xhhv-38mg"	Unique upstream identifier tracking[1]
raw.html_url	evidence.source_url	"https://github.com/advisories/GHSA-q8m4-xhhv-38mg"	Canonical source URL for provenance[1]
N/A (System Assigned)	evidence.evidence_type	"SECURITY_ADVISORY"	Categorical typing for normalization rules[1]
raw.published_at	evidence.provenance.pub_date	"2026-03-20T20:48:14Z"	Historical timeline anchoring[1]
raw.summary	evidence.normalized_payload.summary	"etcd: Authorization bypasses in multiple APIs"	Human-readable belief context[1]
raw.affected[0].ranges	evidence.normalized_payload.range	">=3.5.0, <3.5.28"	Version bound evaluation[1]
raw.affected[0].fixed_in	evidence.normalized_payload.fix_ver	"3.5.28"	Remediation version binding[1]
Canonical JSON String	evidence.content_hash	SHA256("...")	Cryptographic idempotency & replay check[1]

Evidence → Belief Mapping

Normalized evidence records derived from raw inputs generate precise semantic claims within the belief table and relational edges within belief_edge[1]. The table below outlines the exact mapping of derived beliefs, claim types, subjects, predicates, objects, and their supporting evidence bindings[1].

Derived Belief ID	Claim Subject	Predicate	Object	Claim Type	Supporting Evidence ID
bel_v3527_v	etcd:v3.5.27	HAS_VULNE	CVE-2026-	VULNERABI	ev_ghsa_q8m4_xhhv_3

ulnerable		RABILITY	33413	LITY_STATE	8mg [cite: 1, 2]
bel_v3528_patches_cve	etcd:v3.5.28	PROVIDES_FIX_FOR	CVE-2026-33413	REMEDIATION_STATE	ev_rel_v3528 [cite: 1, 2]
bel_v350_corrupted	etcd:v3.5.0	EXHIBITS_FLAW	DATA_INCONSISTENCY	RELIABILITY_STATE	ev_postmortem_v35 [cite: 1, 4]

The resulting belief topology forms a directed acyclic graph[1]. Security evidence (ev_ghsa_q8m4_xhhv_38mg) supports the derived vulnerability claim bel_v3527_vulnerable via a SUPPORTS edge in belief_edge[1]. To prevent premature or unsafe automation, bel_v3527_vulnerable cannot be promoted independently; it establishes a dependency on bel_v3528_patches_cve, which is supported by release evidence ev_rel_v3528[1]. Only when both supporting evidence branches are present does the graph unlock the promotion gate[1].

Evidence → Debt Mapping

A fundamental pattern in Solvent's architecture is the accumulation and retirement of evidence debt[1]. Beliefs derived from incomplete evidence accumulate debt items that block status promotion until additional confirming evidence is ingested[1].

The dataset defines two specific debt mechanisms:

1. **Remediation Fix Availability Debt**: When advisory evidence ev_ghsa_q8m4_xhhv_38mg is ingested, Solvent derives vulnerability belief bel_v3527_vulnerable[1]. However, system safety rules dictate that an action intent to block or replace a vulnerable deployment cannot become LEGAL until a verified patch version exists in the ledger[1]. Solvent attaches a Fix Availability Debt item to bel_v3527_vulnerable[1]. This debt remains active until release evidence ev_rel_v3528 is ingested, confirming that version 3.5.28 is available[1]. Upon ingestion, Solvent resolves the debt item, unlocking the belief for promotion[1].
2. **Vendor Confirmation Debt**: When raw issue reports (#13654) alleging data corruption are ingested, Solvent derives reliability claim bel_v350_corrupted[1]. To prevent automated failover intents from triggering on unverified community reports, Solvent attaches a Vendor Confirmation Debt item requiring official maintainer confirmation[1]. Ingesting official postmortem evidence ev_postmortem_v35 (v3.5-data-inconsistency.md) satisfies this debt requirement, allowing the system to safely promote the reliability claim and execute necessary remediation intents[1].

Normal Path (Scenario A)

The normal path demonstrates the complete lifecycle of evidence ingestion, debt accumulation, debt retirement, belief promotion, and action intent gating under nominal operating conditions[1].
The scenario progresses through six sequential operations:

1. **Initial Evidence Ingestion**: The CLI ingests security advisory evidence ev_ghsa_q8m4_xhhv_38mg and release metadata ev_rel_v3527[1].
2. **Belief and Intent Initialization**: Solvent derives belief bel_v3527_vulnerable ("etcd v3.5.27 contains CVE-2026-33413") with an initial status of DERIVED[1]. Solvent automatically attaches an outstanding Fix Availability Debt item to the belief and creates an associated action intent act_block_v3527 ("Block deployment of etcd v3.5.27") initialized in PENDING status[1].
3. **Remediation Ingestion**: The CLI ingests official release metadata ev_rel_v3528[1].
4. **Debt Satisfaction**: Solvent processes ev_rel_v3528, derives remediation belief bel_v3528_patches_cve, and satisfies the Fix Availability Debt item attached to bel_v3527_vulnerable[1].
5. **Transactional Promotion**: With all debt retired, Solvent executes an atomic CockroachDB transaction that updates bel_v3527_vulnerable status from DERIVED to PROMOTED[1].
6. **Intent Gate Transition**: Within the same database transaction, the promotion trigger evaluates action gating rules and updates act_block_v3527 status from PENDING to LEGAL, authorizing downstream infrastructure controllers to execute the block[1].

Replay Path (Scenario B)

The replay path proves that Solvent's normalization pipeline, cryptographic hashing, and belief derivation routines are completely deterministic and idempotent[1].
The deterministic replay sequence follows four steps:

1. **Re-ingestion Trigger**: The CLI is instructed to re-ingest the exact same offline snapshot file (etcd_demo_fixture.json) that was processed during Scenario A[1].
2. **Cryptographic Content Matching**: For every record in the snapshot, Solvent canonicalizes the raw payload and computes its SHA-256 content hash[1]. The ingestion engine queries the evidence table and matches the computed hashes against existing records[1].
3. **Mutation Suppression**: Solvent identifies that all evidence records already exist[1]. The pipeline suppresses duplicate database inserts into evidence, skips redundant belief derivations in belief, and prevents duplicate edge creation in belief_edge[1].
4. **State Verification**: A database verification check confirms that CockroachDB row counts, belief statuses, and action intent states remain unchanged[1]. No duplicate intents, orphaned edges, or state mutations are generated[1].

Contradiction Path (Scenario C)

In real-world security operations, authoritative vulnerability feeds almost never publish directly contradictory statements within the same ingestion window[1]. Instead, security advisories evolve sequentially over time[1]. To present a clean Contradiction Path (Scenario C) without inventing false historical facts, Solvent incorporates a single, explicitly labeled synthetic fixture record (ev_synth_vendor_override) representing a conflicting vendor assertion[1].
The contradiction workflow proceeds through four operational steps:

1. **Promoted Baseline**: Belief bel_v3527_vulnerable exists in PROMOTED status, and its action intent act_block_v3527 is LEGAL[1].
2. **Conflicting Evidence Ingestion**: The CLI ingests ev_synth_vendor_override, an explicitly demarcated synthetic record claiming that etcd build variant v3.5.27-custom includes backported authorization fixes and is unaffected by CVE-2026-33413[1].
3. **Conflict Edge Derivation**: Solvent normalizes the override record and detects a direct logical contradiction with bel_v3527_vulnerable[1]. Solvent writes a CONTRADICTS edge into the belief_edge table linking the two claims[1].
4. **Gating Lock Execution**: The presence of an active, unresolved CONTRADICTS edge automatically freezes belief promotion rules[1]. Solvent prevents act_block_v3527 from transitioning to EXECUTED, locking the action intent until human operators resolve the conflicting evidence[1].

Falsification / Retraction Path (Scenario D)

Scenario D demonstrates Solvent's core architectural thesis: "Memory is refusing to act on what is no longer true"[1]. Using 100% authentic etcd maintainer postmortems, this path demonstrates automated, cascading safety retractions when previously accepted beliefs are invalidated by new facts[1].
The falsification workflow unfolds across four steps:

1. **Initial Promoted State**: Baseline release evidence ev_rel_v350 was previously ingested, deriving belief bel_v350_stable ("etcd v3.5.0 is General Availability and suitable for production deployments")[1]. The belief was promoted, rendering action intent act_deploy_v350 ("Deploy etcd v3.5.0 cluster") LEGAL[1].
2. **Invalidating Evidence Arrival**: The CLI ingests ev_postmortem_v35 (v3.5-data-inconsistency.md), an official postmortem documenting that v3.5.0 suffers from silent data corruption caused by non-atomic consistent index updates during node crashes[1].
3. **Transactional RetractCascade Execution**: Solvent initiates a single, atomic CockroachDB transaction[1]:
    - Writes an INVALIDATES edge into belief_edge connecting ev_postmortem_v35 to bel_v350_stable[1].
    - Updates bel_v350_stable status from PROMOTED to RETRACTED[1].
    - Executes a recursive graph traversal (RetractCascade), retracting all child beliefs derived from bel_v350_stable[1].
    - Updates action intent act_deploy_v350 status from LEGAL to CANCELLED,

blocking automated deployment pipelines[1].

4. **Audit Trail Preservation**: Although the deployment action was successfully intercepted and revoked, CockroachDB retains an immutable, historical record of all raw evidence, derived belief transitions, and retraction edges, providing full audit compliance[1].

Snapshot / Fixture Recommendation

To guarantee offline evaluation, absolute deterministic replay, and zero dependency on external network APIs, all selected real-world records must be pinned as static JSON fixtures checked directly into the repository[1].

The file organization within the Solvent repository is structured as follows[1]:

- fixtures/etcd_realworld/manifest.json: Contains fixture manifest metadata, record counts, and global cryptographic hashes[1].
- fixtures/etcd_realworld/ATTRIBUTION.md: Provides explicit open-source licensing notices and upstream attributions[8].
- fixtures/etcd_realworld/raw/ghsa_q8m4_xhhv_38mg.json: Stores the unmodified raw JSON payload retrieved from the GitHub Advisory API[1].
- fixtures/etcd_realworld/raw/postmortem_v3.5.json: Stores the pre-parsed JSON representation of the official etcd maintainer postmortem[1].
- fixtures/etcd_realworld/raw/releases_v3.5.json: Stores raw release metadata objects for etcd versions v3.5.0, v3.5.27, and v3.5.28[1].
- fixtures/etcd_realworld/etcd_demo_fixture.json: The single, deterministically ordered snapshot file ingested by Solvent during demo execution[1].

During demonstration runs, Solvent operates entirely against etcd_demo_fixture.json, ensuring fast, deterministic, and network-isolated execution[1].

Licensing / Redistribution Assessment

All primary records selected for the offline snapshot are legally safe for public repository check-in and open-source redistribution[1]. The table below details the intellectual property owners, governing licenses, and compliance requirements for each asset[1].

Source Asset	Intellectual Property Owner	Governing License	Redistribution Terms & Permissibility
GitHub Security Advisories	GitHub, Inc. / Community Contributors	CC-BY-4.0[8]	**Fully Permissible**: Requires attribution notice preserving original URL and author credits[8].

etcd Postmortems & Docs	Cloud Native Computing Foundation (CNCF) / etcd Authors	Apache License 2.0	**Fully Permissible**: Open-source documentation; permissible to quote and redistribute with copyright notice.
etcd Release Metadata	CNCF / etcd Authors	Apache License 2.0	**Fully Permissible**: Standard metadata format freely re-licensable for evaluation.

To maintain full legal compliance, an ATTRIBUTION.md file must be placed alongside the snapshot fixtures in the repository[1]. This file explicitly credits the GitHub Advisory Database under CC-BY-4.0 and the CNCF etcd project under Apache-2.0[8].

Why etcd Is a Good Demonstration Domain

etcd provides a compelling demonstration domain for transactional belief management because it combines high operational stakes with a compact, understandable data lifecycle[1]. As the core distributed state store for Kubernetes, etcd holds all cluster configuration, secret, and workload state[2]. Security vulnerabilities or data corruption defects in etcd directly threaten the availability and integrity of entire cloud environments[2]. Demonstrating automated belief gating and retraction on etcd data immediately conveys real-world value to technical evaluators[1].

Furthermore, software release and vulnerability lifecycles present a compact state space[1]. A complete lifecycle—encompassing vulnerability disclosure, version binding, patch release, and maintainer postmortem—can be fully demonstrated using fewer than ten records[1]. This allows evaluators to trace every state transition manually without becoming overwhelmed by massive datasets[1].

Finally, database upgrade decisions demand absolute transactional certainty[1]. Executing an upgrade based on stale or invalidated security assumptions can result in severe cluster outage or permanent data loss[1]. Showing how Solvent's RetractCascade automatically intercepts and cancels an unsafe deployment intent when invalidating evidence arrives highlights the essential power of a transactional belief ledger[1].

Generalization Beyond Cybersecurity

The core pattern implemented in Solvent—normalizing raw evidence, deriving semantic beliefs, accumulating debt, and gating action intents behind transactional retractions—is

fundamentally domain-agnostic[1]. The identical four-table kernel applies directly to critical decision workflows across multiple industries[1].

In cloud infrastructure operations, raw health probe telemetry and drift detection events serve as evidence[1]. From this evidence, the system derives beliefs regarding regional availability zone degradation[1]. Fix confirmation debt requires independent secondary probes to verify outages before failover action intents are rendered legal[1]. If primary region telemetry recovers before failover completes, new health evidence triggers a retraction cascade, safely cancelling the failover intent before DNS routing is modified[1].

In financial settlement and escrow management, SWIFT payment confirmations and clearing house ledger feeds act as evidence[1]. Solvent derives beliefs regarding transaction clearance, accumulating double-entry reconciliation debt[1]. Once reconciliation evidence arrives and debt is retired, the belief is promoted, authorizing an action intent to release collateral from escrow[1]. If the clearing house issues an automated clawback notice due to fraud detection, the clawback record acts as invalidating evidence, triggering RetractCascade to immediately revoke the escrow release intent before funds exit the ledger[1].

Rejected Alternatives

To adhere strictly to Solvent's frozen kernel constraints and optimize for hackathon demonstration clarity, several candidate technologies and dataset designs were evaluated and rejected[1]. The table below summarizes these rejected options and provides the technical justification for their exclusion[1].

Rejected Alternative	Category	Technical Justification for Rejection
Live NVD / OSV REST APIs	Infrastructure	Rejected due to network non-determinism, API rate limits, and latency during evaluations[1].
Vector Databases / RAG / Embeddings	Architecture	Rejected; Solvent is a deterministic transactional belief ledger, not an approximate semantic search engine[1].
Kafka / Real-time Streaming	Message Bus	Rejected; adds unnecessary operational complexity for a fixed local benchmark dataset[1].

Full NVD CVE Dataset (>200k records)	Dataset	Rejected; massive datasets obscure the core demo narrative and make step-by-step verification impossible[1].
Schema Redesign (Adding 5th Table)	Schema	Rejected; Solvent's four-table CockroachDB schema is frozen and fully sufficient[1].

Risks and Unknowns

Two minor technical edge cases must be managed during dataset integration:
First, string formatting variations in semantic version numbers represent a potential parsing risk[1]. Official etcd release tags utilize a leading v prefix (v3.5.28), whereas security advisory feeds frequently specify versions without prefixes (3.5.28)[2]. If unhandled, string mismatches would break exact-match joins across the evidence and belief tables[1]. To eliminate this risk, Solvent's normalization logic must explicitly strip string prefixes and parse all versions into standard semver tuples prior to database insertion[1].
Second, concurrent database transactions during rapid retraction cascades could theoretically encounter serialization conflicts under heavy thread contention in CockroachDB[1]. To guarantee absolute execution stability during live judge evaluations, the demonstration CLI runner must execute scenario mutations sequentially within explicit BEGIN TRANSACTION ... COMMIT blocks[1].

Exact Implementation Specification for Coding Agent

This specification provides the concrete integration contract for the coding agent responsible for integrating the offline real-world etcd dataset into Solvent[1].
The coding agent must create the following file layout within the repository[1]:

- solvent/fixtures/etcd_realworld/manifest.json
  [cite: 1]
- solvent/fixtures/etcd_realworld/ATTRIBUTION.md
  [cite: 8, 13]
- solvent/fixtures/etcd_realworld/etcd_demo_fixture.json
  [cite: 1]

The pinned JSON snapshot (etcd_demo_fixture.json) must contain an array of normalized evidence objects formatted according to the exact JSON structure below[1]:

JSON


```
[
 {
  "evidence_id": "ev_ghsa_q8m4_xhhv_38mg",
  "source_url": "https://github.com/advisories/GHSA-q8m4-xhhv-38mg",
  "source_id": "GHSA-q8m4-xhhv-38mg",
  "source_type": "GITHUB_ADVISORY",
  "retrieval_date": "2026-08-01T00:00:00Z",
  "publication_date": "2026-03-20T20:48:14Z",
  "raw_payload": {
   "cve_id": "CVE-2026-33413",
   "summary": "etcd: Authorization bypasses in multiple APIs",
   "severity": "HIGH",
   "affected_range": ">=3.5.0, <3.5.28",
   "fixed_version": "3.5.28"
  }
 },
 {
  "evidence_id": "ev_rel_v3527",
  "source_url": "https://github.com/etcd-io/etcd/releases/tag/v3.5.27",
  "source_id": "v3.5.27",
  "source_type": "RELEASE_METADATA",
  "retrieval_date": "2026-08-01T00:00:00Z",
  "publication_date": "2026-02-10T00:00:00Z",
  "raw_payload": {
   "version": "3.5.27",
   "is_prerelease": false
  }
 },
 {
  "evidence_id": "ev_rel_v3528",
  "source_url": "https://github.com/etcd-io/etcd/releases/tag/v3.5.28",
  "source_id": "v3.5.28",
  "source_type": "RELEASE_METADATA",
  "retrieval_date": "2026-08-01T00:00:00Z",
  "publication_date": "2026-03-20T00:00:00Z",
  "raw_payload": {
   "version": "3.5.28",
   "is_prerelease": false
  }
 },
 {
  "evidence_id": "ev_postmortem_v35",
```

```
   "source_url":
"https://github.com/etcd-io/etcd/blob/main/Documentation/postmortems/v3.5-data-inconsistency.md",
   "source_id": "POSTMORTEM-v3.5-CORRUPT",
   "source_type": "POSTMORTEM",
   "retrieval_date": "2026-08-01T00:00:00Z",
   "publication_date": "2022-04-20T00:00:00Z",
   "raw_payload": {
    "affected_versions": ["3.5.0", "3.5.1", "3.5.2"],
    "fixed_version": "3.5.3",
    "issue_keys": ["#13514", "#13654", "#13766"]
   }
  },
  {
   "evidence_id": "ev_synth_vendor_override",
   "source_url": "https://synthetic.local/advisories/OVERRIDE-2026-01",
   "source_id": "SYNTH-OVERRIDE-01",
   "source_type": "SYNTHETIC_OVERRIDE",
   "retrieval_date": "2026-08-01T00:00:00Z",
   "publication_date": "2026-03-21T00:00:00Z",
   "is_synthetic": true,
   "raw_payload": {
    "target_version": "3.5.27",
    "asserts_unaffected": true,
    "cve_id": "CVE-2026-33413"
   }
  }
]
```


The coding agent must map this fixture payload across Solvent's four database tables as follows[1]:

1. Table evidence: Maps id from evidence_id, source_url from source_url, source_id from source_id, evidence_type from source_type, raw_payload from stringified raw_payload, and content_hash from the computed SHA256 hash of the canonical JSON string[1].
2. Table belief: Maps derived claims including bel_v3527_vulnerable (subject = 'etcd:3.5.27', predicate = 'HAS_VULNERABILITY', object = 'CVE-2026-33413', status = 'DERIVED') and bel_v3528_patches_cve (subject = 'etcd:3.5.28', predicate = 'PROVIDES_FIX_FOR', object = 'CVE-2026-33413', status = 'PROMOTED')[1].
3. Table action_intent: Maps act_block_v3527 (action_type = 'BLOCK_DEPLOYMENT', target = 'cluster:production-etcd-v3.5.27', status = 'PENDING') bound to bel_v3527_vulnerable[1].
4. Table belief_edge: Maps invalidation edges including edge_retract_v350 (parent_belief_id = 'ev_postmortem_v35', child_belief_id = 'bel_v350_stable', edge_type = 'INVALIDATES')[1].

Acceptance Criteria

The integration must satisfy five automated end-to-end acceptance tests prior to demonstration approval[1]. The table below outlines the command triggers, expected system mechanics, and verification criteria for each test suite[1].

Test Suite	Command Trigger	Expected System Mechanics	Verification Criteria
1. Offline Ingestion	solvent ingest --fixture etcd_demo_fixture.json --offline [cite: 1]	Ingests snapshot fixture without initiating network calls[1].	System exits with code 0. Parses and writes all 5 evidence records[1].
2. Replay Determinism	Re-run ingestion command twice in succession[1]	Computes SHA-256 content hashes and matches existing database rows[1].	Zero duplicate rows inserted. Table record counts remain identical[1].
3. Debt-Gated Promotion	solvent promote --belief bel_v3527_vulnerable [cite: 1]	Verifies fix release debt satisfaction and executes promotion transaction[1].	Belief status updates to PROMOTED. Action intent act_block_v3527 transitions to LEGAL[1].
4. Transactional Retraction	solvent retract --evidence ev_postmortem_v35 [cite: 1, 4]	Triggers RetractCascade in CockroachDB transaction upon postmortem ingestion[1].	Belief status updates to RETRACTED. Action intent act_deploy_v350 transitions to CANCELLED[1].
5. Audit Trail Verification	solvent audit --intent act_deploy_v350 [cite: 1]	Queries CockroachDB audit trail for intent history[1].	Displays full provenance trace showing ingestion, promotion,

			invalidation, and cancellation[1].

Works cited

1. research_prompt.md
2. etcd: Authorization bypasses in multiple APIs · CVE-2026-33413 - GitHub, https://github.com/advisories/GHSA-q8m4-xhhv-38mg
3. CHANGELOG-3.5.md - etcd-io/etcd - GitHub, https://github.com/etcd-io/etcd/blob/main/CHANGELOG/CHANGELOG-3.5.md
4. etcd/Documentation/postmortems/v3.5-data-inconsistency.md at main - GitHub, https://github.com/etcd-io/etcd/blob/main/Documentation/postmortems/v3.5-data-inconsistency.md
5. GHSA-q8m4-xhhv-38mg - OSV - Open Source Vulnerabilities, https://test.osv.dev/vulnerability/GHSA-q8m4-xhhv-38mg
6. etcd revision occurs Inconsistent · Issue #13654 - GitHub, https://github.com/etcd-io/etcd/issues/13654
7. CVE-2026-33413 Detail - NVD, https://nvd.nist.gov/vuln/detail/cve-2026-33413
8. GitHub Advisory Database, https://github.com/advisories?query=credit%3Ah3ri0s
9. etcd: Watch API authorization bypass via open-ended range requests · GHSA-xg4h-6gfc-h4m8 - GitHub, https://github.com/advisories/GHSA-xg4h-6gfc-h4m8
10. GHSA-2xhq-gv6c-p224 - OSV, https://osv.dev/GHSA-2xhq-gv6c-p224
11. Etcd Gateway can include itself as an endpoint resulting in resource exhaustion · CVE-2020-15114 - GitHub, https://github.com/advisories/GHSA-2xhq-gv6c-p224
12. etcd RBAC bypass allows unauthorized data access via PrevKv/lease attachment in nested transaction Put requests - GitHub, https://github.com/advisories/GHSA-x35m-3gp4-4fh5
13. github/advisory-database: Security vulnerability database inclusive of CVEs and GitHub originated security advisories from the world of open source software., https://github.com/github/advisory-database